Veritas Tutorial Assignment Report

Correct-by-Construction Verilog Generation via CNF -> BENCH -> Verilog

Submitted to: Prof. Dr. Ramesh Karri and Weihua Xiao

Name: Tejas Attarde

Document: Veritas_Report.pdf

Contents

- 1. Assignment Scope and Workflow
- 2. Experimental Setup and Pipeline Notes
- 3. Summary of the Three Required Configurations
- 4. Run 1: 3-bit Adder
- 5. Run 2: 2x4 Decoder
- 6. Different-Model Comparison: Gemini 2.5 Flash on the 3-bit Adder
- 7. Run 3 / Part IV: 5-bit Adder Scale-Up
- 8. Consolidated Oracle Validation Summary
- 9. Reflection: Why CNF Is a Useful Intermediate Representation
- 10. Conclusion
- Appendix A. Generated Filenames by Run
- Appendix B. Prompt Refinement and Debugging Notes
- Appendix C. Final Artifact Inventory

The three reported configurations are the 3-bit adder, the 2x4 decoder, and the 5-bit adder scale-up. The Gemini experiment reuses the 3-bit adder configuration, so I discuss it separately as a controlled comparison rather than counting it as a fourth required configuration.

1. Assignment Scope and Workflow

The purpose of this assignment was to use the Veritas flow to generate logic in a way that stays correct by construction instead of depending only on post-generation debugging. Rather than asking directly for Verilog and hoping the result is right, the flow first builds a CNF description of the circuit, converts that CNF into BENCH, simulates the BENCH circuit to obtain a truth table, and then translates the BENCH netlist into synthesizable Verilog. That order matters because the logical behavior becomes explicit before the final HDL is produced.

I organized my work in four stages. First, I completed two baseline runs as required in Part I. I chose a 3-bit adder because it is large enough to show the ripple-carry pattern clearly while still being

small enough to inspect by hand, and I chose a 2x4 decoder because the assignment required at least one non-adder design. Second, I wrote Python oracles for both baseline runs and checked the generated truth tables row by row. Third, I repeated the 3-bit adder run with Gemini 2.5 Flash so I could compare the same configuration under a different model. Finally, I scaled the adder family from 3 bits to 5 bits and used the same style of validation to see how the flow behaved at a larger size.

A practical lesson from the notebook was that correctness in this workflow depends on naming discipline as much as it depends on logic. The variables declared in the CNF must match the input and output names given to the CNF-to-BENCH conversion step. If the names drift, a logically sensible CNF can still fail later in the pipeline. Because of that, I treated variable lists as part of the experiment design rather than as a cosmetic detail.

This approach also encouraged a more careful engineering workflow. I started with a smaller design, verified it rigorously, and only then scaled up. That mattered most in the 5-bit adder experiment, where the challenge was not the arithmetic itself but keeping the equations fully expanded and parser-friendly. Starting from a known-good smaller run made it much easier to see which changes came from the larger size and which changes came from formatting.

2. Experimental Setup and Pipeline Notes

All experiments used the same notebook structure. After setting the design configuration, the notebook generated CNF, converted that CNF to BENCH using explicit input and output names, simulated the BENCH netlist to obtain truth-table artifacts, and then converted the BENCH file into Verilog. I treated each of those stages as a checkpoint. Before moving on, I checked the equations, the naming, the artifact files, and the row counts.

For the adder family, I used ripple-carry naming. Inputs were ordered from least significant bit to most significant bit and were followed by a carry-in term. Visible outputs were the sum bits plus the final carry-out. Intermediate carries such as C1, C2, C3, and C4 appeared in the CNF but did not need to be declared as final outputs in the BENCH conversion step. This kept the truth-table comparison clean because the oracle only had to compare the externally visible outputs.

For the decoder, I used the simple two-input convention A and B with one-hot outputs Y0 through Y3. In that case the correctness property was not numeric addition but exact one-hot behavior. Every input combination had to activate one and only one output. Because the truth table had only four rows, the decoder also served as a useful sanity check that the pipeline handled different styles of combinational logic cleanly.

The alternate-model comparison introduced a different kind of issue. The logic itself was still usable, but the returned equations were more verbose and came with different carry names. I normalized those names before running the downstream conversion and validation steps. The fact that the design still passed the oracle after normalization was useful evidence that the pipeline depends on the correctness of the representation, not on one specific output style.

Stage	Main check	Reason

CNF generation	Equation count, variable names, explicit output definitions	Confirms that the logic specification is readable and complete before conversion.
CNF -> BENCH	Declared inputs and outputs match the CNF exactly	Prevents parser and naming mismatches.
BENCH simulation	Truth-table files exist and row counts match the input space	Shows that the generated netlist can be exercised exhaustively.
BENCH -> Verilog	Output file written successfully with preserved naming	Confirms a clean final translation to HDL.
Oracle validation	Every row matches the reference function	Provides behavioral evidence that the generated design is correct.

Treating each stage as a checkpoint made the notebook much easier to manage. Instead of running the whole workflow blindly and debugging only at the end, I could localize issues quickly. That same habit made the final report easier to write because each section already had a clear set of artifacts and validation results attached to it.

3. Summary of the Three Required Configurations

The assignment requires three reported configurations overall. In my report, I count the two Part I baseline designs and the larger Part IV scale-up design as the three required configurations. I treat the Gemini section as a comparison because it reuses the 3-bit adder configuration rather than introducing a new design family or size.

Run	Design	Type	Primary inputs	Primary outputs	Validation result
Run 1	adder	3-bit	A0, B0, A1, B1, A2, B2, Cin	S0, S1, S2, Cout	PASS (128/128 rows)
Run 2	decoder	2x4	A, B	Y0, Y1, Y2, Y3	PASS (4/4 rows)
Run 3	adder	5-bit	A0, B0, A1, B1, A2, B2, A3, B3, A4, B4, Cin	S0, S1, S2, S3, S4, Cout	PASS (2048/2048 rows)

I chose this set deliberately. The 3-bit adder is a clear arithmetic baseline, the 2x4 decoder demonstrates that the workflow also handles one-hot control logic, and the 5-bit adder lets me discuss scale-up behavior without switching to a completely different family. Keeping the larger run in the same family made the Part IV comparison much more meaningful.

Another reason this combination worked well is that each run contributes something different. The first adder run established the basic ripple-carry pattern, the decoder run checked a non-arithmetic

function with a small and inspectable truth table, and the 5-bit adder showed how equation count, clause count, runtime, and exhaustive truth-table size grow when the design gets larger.

4. Run 1: 3-bit Adder

The first complete baseline run was a 3-bit ripple-carry adder. I chose this design because it balances clarity and complexity. It is large enough to contain internal carries and multiple stages, but still compact enough that the resulting equations can be inspected directly when needed.

Field	**Run 1 value**
Design / type	adder / 3-bit
Input naming	A0, B0, A1, B1, A2, B2, Cin
Output naming	S0, S1, S2, Cout
Internal signals visible in CNF	C1, C2
Expected truth-table size	2^7 = 128 rows
Model used	gpt-5-mini-2025-08-07

The generated CNF followed the ripple-carry structure I expected. The first stage defined S0 and C1 from A0, B0, and Cin. The second stage used A1, B1, and C1 to define S1 and C2. The last stage used A2, B2, and C2 to define S2 and Cout. Because the structure was incremental, it was easy to compare the equations against the intended arithmetic.

For the conversion step, I declared the input list exactly as A0, B0, A1, B1, A2, B2, Cin and the output list as S0, S1, S2, Cout. That matched the equations and prevented ambiguity during CNF-to-BENCH translation. After conversion, the notebook produced both the standard CSV truth-table file and the tabular CSV used later in validation. The BENCH-to-Verilog step also completed successfully.

Generated filenames for Run 1: adder_3-bit.cnf, adder_3-bit.bench, adder_3-bit.csv, adder_3-bit_tab.csv, and adder_3-bit.v.

The oracle for this run reconstructed the integers A and B from the bit columns and then computed result = A + B + Cin. It extracted S0, S1, S2, and Cout from that result by bit slicing. Every one of the 128 rows matched the oracle, so this run gave me a strong baseline for the rest of the assignment.

What I found most useful about this run is that it gave two kinds of confidence at once. Structurally, the equations looked like a sensible ripple-carry chain. Behaviorally, the truth table matched the arithmetic oracle on the full input space. Together those checks were more convincing than a single HDL file by itself would have been.

5. Run 2: 2x4 Decoder

My second baseline run was a 2x4 decoder, which satisfied the requirement to include at least one non-adder design. I wanted a control-style circuit here because it behaves very differently from an adder. Instead of propagating carries and producing numeric outputs, the decoder maps a two-bit input to a one-hot output vector.

Field	**Run 2 value**
Design / type	decoder / 2x4
Input naming	A, B
Output naming	Y0, Y1, Y2, Y3
Expected truth-table size	2^2 = 4 rows
Model used	gpt-5-mini-2025-08-07

The decoder equations expressed each one-hot output as a constrained relationship between the two inputs and the corresponding output line. Functionally, the behavior was Y0 for 00, Y1 for 01, Y2 for 10, and Y3 for 11. That made the validation rule simple: every input should activate exactly one output line and leave the others low.

For the conversion step, I used A and B as inputs and Y0 through Y3 as outputs. Keeping those names aligned with the generated equations was important because decoder naming can shift across different prompt attempts. Once the names were fixed, the CNF-to-BENCH and BENCH-to-Verilog stages completed cleanly.

Generated filenames for Run 2: decoder_2x4.cnf, decoder_2x4.bench, decoder_2x4.csv, decoder_2x4_tab.csv, and decoder_2x4.v.

The decoder oracle checked the expected one-hot index for each row and compared it with the generated outputs. The final result was a full pass: all 4 of the 4 simulated rows matched. This run was useful because it showed that the workflow also handles exact combinational control behavior and not only arithmetic datapaths.

This run also reinforced the naming lesson from the adder experiment. Once the converter inputs and outputs were aligned with the equation names, the rest of the flow became predictable. That same lesson carried directly into the comparison section and the 5-bit scale-up.

6. Different-Model Comparison: Gemini 2.5 Flash on the 3-bit Adder

To satisfy the alternate-model requirement, I reran the 3-bit adder using Gemini 2.5 Flash. I deliberately reused the same adder configuration from Run 1 so that the comparison would stay controlled. If both the design and the model had changed at the same time, it would have been much harder to attribute any differences in behavior.

Field	Comparison result
Base configuration reused	adder / 3-bit
Model used	gemini-2.5-flash
Truth-table file used for oracle	adder_gemini_3-bit_tab.csv
Observed naming difference	Cin0 and Cout2 instead of Cin and Cout
Validation result	PASS (128/128 rows)

The main difference I saw was not logical correctness but formatting and naming style. Gemini returned equations in a more verbose form and used a different carry naming convention. I normalized those names before sending the equations into the parser and converter. That cleanup did not change the function of the circuit; it only made the representation compatible with the rest of the workflow.

The artifact naming in this section preserved both a raw CNF export and a cleaned CNF export. The raw file is adder_3-bit_gemini_raw.cnf and the cleaned CNF used for the downstream flow is adder_3-bit_gemini.cnf. The BENCH, CSV, tabular CSV, and Verilog files then follow the adder_gemini_3-bit.* naming convention: adder_gemini_3-bit.bench, adder_gemini_3-bit.csv, adder_gemini_3-bit_tab.csv, and adder_gemini_3-bit.v.

After normalization, the Gemini-generated adder converted to BENCH, simulated correctly, and passed the oracle on all 128 rows. That result was important because it showed that the workflow is not tied to one specific model family or one exact surface style of equations. The critical requirement is that the logical content remains correct and can be normalized into the format expected by the parser and simulator.

Because this comparison reused the same 3-bit adder configuration, I do not count it as one of the three required configurations in the report. I treat it as a controlled comparison against Run 1. That separation makes it easier to distinguish design complexity from output-style differences.

7. Run 3 / Part IV: 5-bit Adder Scale-Up

For the scale-up experiment, I extended the adder family from 3 bits to 5 bits. I chose this direction because it preserved the ripple-carry structure while making the design meaningfully larger. That allowed me to measure how the CNF and runtime changed without switching to a different circuit family.

Field	Run 3 value
Design / type	adder / 5-bit
Input naming	A0, B0, A1, B1, A2, B2, A3, B3, A4, B4, Cin
Output naming	S0, S1, S2, S3, S4, Cout

Internal signals visible in CNF	C1, C2, C3, C4
Expected truth-table size	2^11 = 2048 rows
Model used	gpt-5-mini-2025-08-07

The main issue in this run was not the arithmetic itself but the format of the generated equations. For larger designs, the response tended to drift toward compact indexed notation. That style may look concise, but it is not ideal for a parser that expects explicit equations. To address that, I refined the prompt to require all equations in expanded form and to require exact variable names. Once that was done, the generated CNF was compatible with the rest of the flow.

The 5-bit run produced the expected set of equations for S0, C1, S1, C2, S2, C3, S3, C4, S4, and Cout. After I aligned the converter input and output lists with those names, the BENCH generation step succeeded. The simulation stage produced the truth-table artifacts, and the BENCH-to-Verilog stage wrote the final Verilog file.

Generated filenames for Run 3: adder_5-bit.cnf, adder_5-bit.bench, adder_5-bit.csv, adder_5-bit_tab.csv, and adder_5-bit.v.

The oracle for the 5-bit adder reconstructed the two five-bit operands and carry-in, computed the integer sum, and then compared S0 through S4 and Cout against the truth-table outputs. The result was a full pass across all 2048 rows. This was the strongest validation result in the notebook because it combined a larger input space with a larger visible output set.

Metric	**3-bit adder**	**5-bit adder**	**Observed growth**
Primary inputs	7	11	1.57x
Primary outputs	4	6	1.50x
Truth-table rows	128	2048	16x
CNF text length	957 characters	1983 characters	2.1x
Total CNF clauses	42	80	1.9x
Number of equations	6	10	1.7x
Generation time	~10-15 s baseline	66.5 s	substantially higher

The most interesting lesson from the scale-up is that different parts of the workflow grow at different rates. The CNF text size and clause count grew roughly linearly with bit width, which is what I expected for a ripple-carry structure. The truth table, however, grew exponentially because the number of input combinations increased from 2^7 to 2^11. That difference matters in practice: the symbolic description scales gently, but exhaustive behavioral validation scales much faster.

Overall, the 5-bit run confirmed that the pipeline still works at a larger size, but it also showed why prompt discipline becomes more important as complexity increases. Expanded equations, stable

naming, and early validation checks mattered much more here than they did in the smaller baseline run.

8. Consolidated Oracle Validation Summary

The assignment requires pass/fail reporting and mismatch details if any appear. In the final notebook, every required validation passed. Because no mismatches remained in the submitted version, there is no failing input vector to report in the final results table.

Configuration	Oracle method	Rows checked	Result	Mismatch details
adder / 3-bit	Integer addition with carry-in, then bit extraction	128	PASS	None
decoder / 2x4	One-hot output selection from two-bit input index	4	PASS	None
adder / 5-bit	Integer addition with carry-in, then bit extraction	2048	PASS	None
adder / 3-bit via Gemini	Integer addition with carry-in, then bit extraction	128	PASS	None

This table matters because it shows that the generated circuits were not accepted simply because the equations looked plausible. They were accepted because the simulated truth tables matched independently written reference functions across the full input space available in each case. A visually reasonable circuit is not the same thing as a validated circuit.

The oracle stage also made the notebook easier to trust while I was iterating. It separated logic issues from naming issues and parser issues. That separation was one of the main reasons the final report can state the results confidently.

9. Reflection: Why CNF Is a Useful Intermediate Representation

CNF is a useful intermediate representation for correctness-by-construction because it sits at a productive point between high-level intent and low-level implementation. It is more explicit and checkable than a plain natural-language description, but it is still more abstract than HDL code. Each output is defined as a logical formula whose meaning can be examined directly, converted systematically, and validated independently. That makes it easier to separate functional correctness from code-generation style.

Another advantage of CNF is that it forces the behavior of each output to be stated declaratively. In a generated Verilog module, a bug can hide inside control flow, syntax, accidental widths, or ordering effects. In CNF, the statement is simpler: for an output to be correct, the clauses must express

exactly the conditions under which that output is true or false. Even when the equations are long, they still describe logic instead of implementation mechanics.

The Veritas flow takes advantage of this by using CNF as a bridge representation. Once the CNF is correct, the BENCH netlist and the final Verilog are produced through deterministic translation steps instead of another open-ended generation step. In other words, the most error-prone phase is pushed earlier into a form that is much easier to verify. The later steps mostly preserve meaning rather than inventing new structure.

From my perspective, the strongest part of this method is that it combines symbolic and behavioral checking. The symbolic side comes from explicit CNF equations and stable naming. The behavioral side comes from simulating the resulting BENCH file and comparing it to an oracle across all rows. Because the same design is seen through both views, the method provides stronger evidence than either view would provide by itself. That is what makes CNF such a useful intermediate representation for this assignment.

10. Conclusion

This homework showed that the Veritas flow is practical and understandable when it is approached systematically. The two baseline runs established that the pipeline works for both arithmetic and non-arithmetic logic. The alternate-model comparison showed that the method is robust to differences in output style as long as the equations can be normalized into the expected format. The scale-up run demonstrated that larger designs remain tractable, although prompt refinement and validation discipline become more important.

My final results were straightforward: the 3-bit adder passed its 128-row oracle check, the 2x4 decoder passed its 4-row oracle check, the Gemini-based 3-bit adder passed all 128 rows after normalization, and the 5-bit adder passed all 2048 rows in the scale-up study. Taken together, these runs support the central claim of the notebook: if the logical specification is made explicit early and is validated before translation, the resulting Verilog can be obtained in a much more controlled way.

The biggest practical lesson I am taking away is that correctness-by-construction is not only about formal logic. It also depends on disciplined interfaces between stages. Stable variable naming, parser-compatible formatting, complete artifact generation, and oracle-based checking all matter. When those pieces are in place, the CNF-first workflow is much easier to trust than a direct jump from prompt to Verilog.

Appendix A. Generated Filenames by Run

For completeness, I list the filenames used in each run in one place below. This appendix mirrors the final artifact naming convention used in my notebook and artifact folder.

Run	Files
Run 1: adder / 3-bit	adder_3-bit.cnf; adder_3-bit.bench; adder_3-bit.csv; adder_3-bit_tab.csv; adder_3-bit.v

Run 2: decoder / 2x4	decoder_2x4.cnf; decoder_2x4.bench; decoder_2x4.csv; decoder_2x4_tab.csv; decoder_2x4.v
Run 3: adder / 5-bit	adder_5-bit.cnf; adder_5-bit.bench; adder_5-bit.csv; adder_5-bit_tab.csv; adder_5-bit.v
Different-model comparison	adder_3-bit_gemini_raw.cnf; adder_3-bit_gemini.cnf; adder_gemini_3-bit.bench; adder_gemini_3-bit.csv; adder_gemini_3-bit_tab.csv; adder_gemini_3-bit.v

The comparison artifacts are listed separately because they come from the alternate-model section rather than from the three primary reported configurations. I kept both the raw and cleaned CNF exports for that comparison because they document the normalization step that was necessary before conversion and validation.

Appendix B. Prompt Refinement and Debugging Notes

I did not need the same amount of prompt control for every run. The 3-bit adder and the 2x4 decoder were relatively easy to keep in a clean form because the equations were short and the naming was compact. The larger adder required more explicit direction. In particular, I had to state that every equation should be written out explicitly, that compressed indexed notation should not be used, and that the exact input and output names had to match the downstream converter.

The alternate-model comparison taught me a different debugging lesson. There, the logic was still usable, but the returned notation was more verbose and stylistically different from what the parser expected. That reinforced the idea that the workflow should always be debugged in layers: first confirm that the functional content is present, then normalize the formatting, then rerun the deterministic steps. Doing it in the opposite order would have made the process much more confusing.

If I were extending this work further, I would keep the same discipline: small baseline first, oracle validation second, then scale-up. I would also standardize artifact naming as early as possible, because that made the final report easier to assemble and reduced ambiguity across sections.

Appendix C. Final Artifact Inventory

The final submission package contains the executed notebook, this report, and the per-run artifact folders. I also removed the naming mismatch that appeared in an earlier draft of the report so that the listed files now match the actual artifact set.

Folder	Expected files present	Notes
artifacts/adder_3-bit	adder_3-bit.cnf, adder_3-bit.bench, adder_3-bit.csv, adder_3-bit_tab.csv, adder_3-bit.v	Baseline arithmetic run.
artifacts/decoder_2x4	decoder_2x4.cnf, decoder_2x4.bench,	Baseline non-adder run.

	decoder_2x4.csv, decoder_2x4_tab.csv, decoder_2x4.v	
artifacts/gemini_adder_3-bit	adder_3-bit_gemini_raw.cnf, adder_3-bit_gemini.cnf, adder_gemini_3-bit.bench, adder_gemini_3-bit.csv, adder_gemini_3-bit_tab.csv, adder_gemini_3-bit.v	Controlled comparison run with normalization step retained.
artifacts/adder_5-bit	adder_5-bit.cnf, adder_5-bit.bench, adder_5-bit.csv, adder_5-bit_tab.csv, adder_5-bit.v	Scale-up run used in Part IV.

This final inventory is useful because it makes the submission easy to audit against the notebook and the assignment handout. Every major run has its own folder, and every folder contains the artifacts needed to trace the flow from CNF through BENCH simulation to the final Verilog output.

Appendix D. Notes on File Organization and Submission Readiness

I organized the final artifact set by run so that each design can be reviewed independently. That makes it easier to compare the notebook output against the corresponding CNF, BENCH, truth-table, and Verilog files without having to search through a single mixed directory.

The arithmetic baseline, non-adder baseline, alternate-model comparison, and scale-up experiment each keep their own artifact naming. The only special case is the Gemini comparison, where I retained both the raw CNF export and the cleaned CNF export so that the normalization step remains visible and reproducible.

Before packaging the final submission, I also checked that the listed filenames in the report matched the actual artifact folders. This was important because an earlier draft used a cleaned Gemini CNF filename that did not line up with the archived files. The updated report fixes that mismatch and now mirrors the final artifact package directly.

From a grading standpoint, this organization makes the work easier to audit. A reviewer can open any run folder and trace the same progression that appears in the notebook: specification, conversion, simulation, validation, and final HDL output. That consistency is one of the reasons I kept the report and artifact inventory detailed rather than reducing them to a short summary.